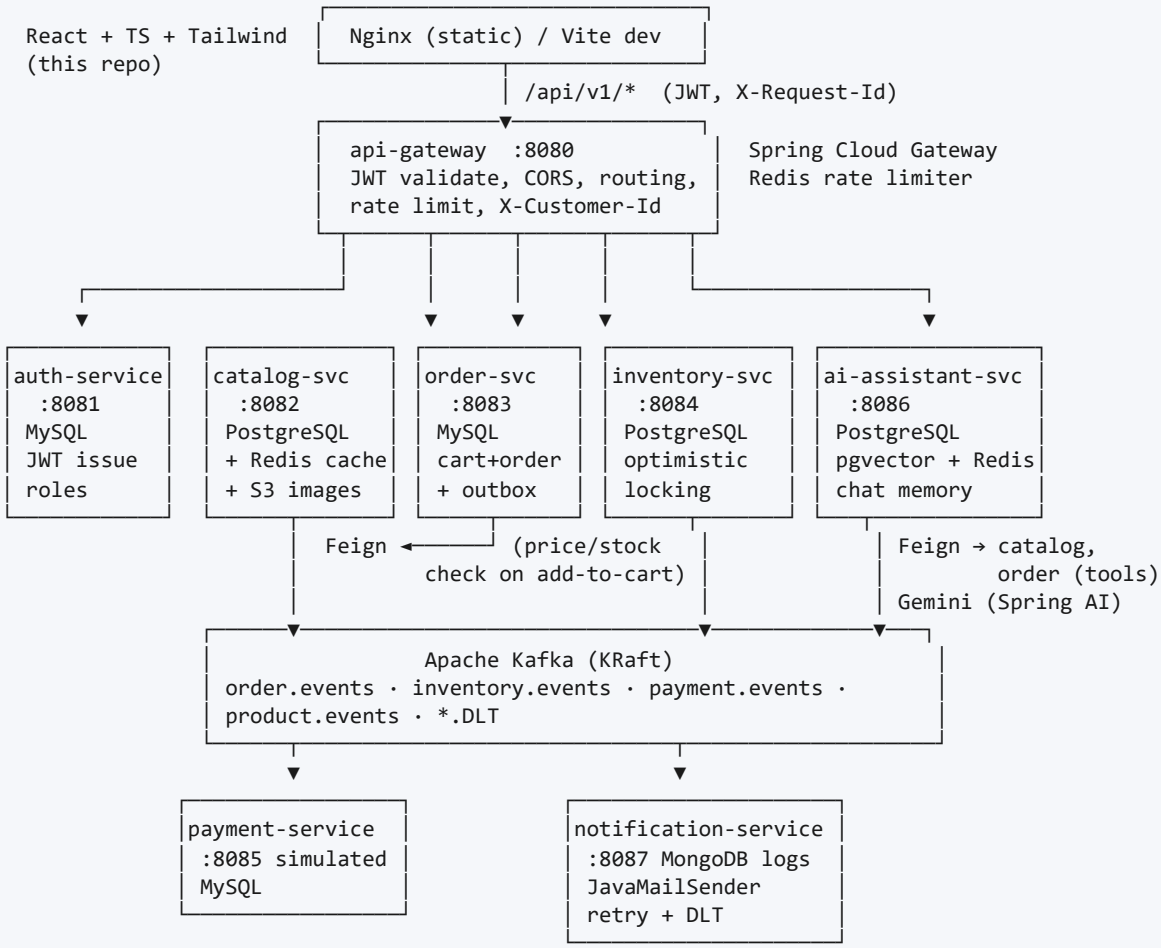
AuraMart — Microservices Architecture

Target backend for the existing React app (`API_CONTRACT.md`). Every service below maps to a skill on the resume.

1. Big picture



Observability: Logback JSON → Elasticsearch → Kibana · Micrometer + OTel → Grafana/Tempo
Platform: Docker Compose (local) · AWS EC2 + S3 + IAM (deploy) · GitHub Actions CI

2. Services

Service	Port	DB	Owns	Key techniques
api-gateway	8080	Redis	Edge	Spring Cloud Gateway, JWT filter, CORS, Redis rate limiter, injects `X-Customer-Id` from token `sub`
auth-service	8081	MySQL	Users, roles	Spring Security, BCrypt, JWT (access + refresh), roles `shopper` / `admin`, Flyway
catalog-service	8082	PostgreSQL	Products, categories	Spring Data JPA/Hibernate, paging & sort, `pg_trgm` typo search, Redis `@Cacheable`/`@CacheEvict`, S3 image upload, `@Version` → `concurrent-update`, publishes `product.events`

Service	Port	DB	Owns	Key techniques
inventory-service	8084	PostgreSQL	Stock	Optimistic locking (`@Version`) to prevent overselling, consumes `OrderPlaced` → `StockReserved`/`StockRejected`, releases on cancel, idempotent consumer
order-service	8083	MySQL	Cart, orders	Feign → catalog (price) & inventory (stock), saga orchestrator, **transactional outbox**, `orderId` partition key, `@Transactional` boundaries
payment-service	8085	MySQL	Payments	Consumes `StockReserved`, simulated success/failure → `PaymentSucceeded`/`PaymentFailed`
notification-service	8087	MongoDB	Notification log	Kafka consumer, `@RetryableTopic` backoff + DLT, idempotency by eventId, JavaMailSender (MailHog locally)
ai-assistant-service	8086	PostgreSQL + pgvector, Redis	Embeddings, chats	Spring AI + Gemini, RAG over products (kept in sync via `product.events`), tool calling (`searchProducts`, `getOrderStatus` via Feign), SSE streaming, conversation memory in Redis, `semantic-search` endpoint

Shared library `common-lib`: RFC 9457 `ProblemDetail` handler, error slugs, event DTOs, `X-Request-Id` MDC filter.

3. Checkout saga (choreography)

```
order-svc       POST /orders → save Order(PENDING) + outbox row (same tx) → OrderPlaced
inventory-svc   OrderPlaced  → reserve (optimistic lock) → StockReserved | StockRejected
order-svc       StockRejected → OrderCancelled(OUT_OF_STOCK)
payment-svc     StockReserved → PaymentSucceeded | PaymentFailed
order-svc       PaymentSucceeded → OrderConfirmed
order-svc       PaymentFailed → OrderCancelled(PAYMENT_FAILED) → inventory releases stock
notification    OrderConfirmed / OrderCancelled → email (retry 3x → DLT)
```

The frontend polls `GET /orders/{id}` every 2 s while `PENDING` (already implemented).

4. Gateway routes

Path	Service
`/api/v1/auth/**`	auth-service
`/api/v1/products/**`	catalog-service (`semantic-search` → ai-assistant-service)
`/api/v1/carts/**`, `/api/v1/orders/**`	order-service
`/api/v1/inventory/**`	inventory-service (admin)
`/api/v1/assistant/**`	ai-assistant-service (SSE, no response buffering)

5. Repository layout

```
auramart/
├── pom.xml                      # parent, Java 21, Spring Boot 3.x, Spring Cloud BOM
├── common-lib/
├── api-gateway/
├── auth-service/
├── catalog-service/
├── inventory-service/
├── order-service/
├── payment-service/
├── notification-service/
├── ai-assistant-service/
├── frontend/                    # this React app
├── infra/
│   ├── docker-compose.yml       # all services + mysql, postgres(pgvector), mongo, redis,
│   │                            # kafka, kafka-ui, mailhog, elasticsearch, kibana
│   └── kibana/
├── .github/workflows/ci.yml     # mvn verify (JUnit 5, Mockito, Testcontainers) + npm test + docker build
└── docs/adr/                    # architecture decision records
```

6. Resume skill coverage

Resume skill	Where
Java 21, Spring Boot, Spring MVC	All services
Spring Data JPA, Hibernate, Flyway	catalog, inventory, order, auth, payment
Spring Security, JWT	auth-service, api-gateway
API Gateway, Feign Client	api-gateway; order → catalog/inventory, ai → catalog/order
Kafka	Saga, outbox, DLT
Redis	Product cache, rate limiting, chat memory
MySQL / PostgreSQL / MongoDB	order+auth+payment / catalog+inventory+ai / notification
pgvector, RAG, Spring AI, Gemini, embeddings	ai-assistant-service
React, TypeScript, Tailwind	Frontend (done)
AWS EC2, S3, IAM	Deploy on EC2, product images on S3, IAM role (no keys in code)
Docker, Docker Compose	Every service has a Dockerfile; one-command `infra/docker-compose.yml`
JUnit 5, Mockito, Postman	Unit + Testcontainers tests, Postman collection in `docs/`
Kibana	Centralized JSON logs searched by `X-Request-Id`
CI/CD, Git, Maven	GitHub Actions

7. Build order

1. Parent POM + `common-lib` + `infra/docker-compose.yml` (databases, Kafka, Redis)
2. catalog-service (CRUD, paging, search, Redis, Flyway) → connect frontend with `VITE_USE_MOCKS=false`
3. auth-service + api-gateway (JWT)
4. order-service (cart, Feign) + inventory-service
5. Kafka saga: outbox, payment-service, compensation
6. notification-service (MongoDB, retry, DLT, email)
7. ai-assistant-service (pgvector RAG, tools, SSE)
8. Tests, Kibana logging, GitHub Actions, AWS EC2 deploy